

Elliptic-curve arithmetic optimizations in this repository

Date: 2026-10-06. Status: survey (no code changes, no new measurements). Base commit: 6350e7f7 (origin/main); every in-repo citation below was verified against that tree. Location: experiments/ec-arithmetic-optimizations/. Visuals: `opt_layers.dot` (layer overview) and `ec_optimizations.dot` (full technique graph), rendered to `figures/*.svg` and `figures/*.pdf`. PDF of this report: `REPORT-2026-10-06.pdf` (built from this file; contains all figures).

Claim labels: **[measured]** (number in a committed doc/receipt on this tree), **[structural]** (design fact verifiable in the cited source), **[conditional]** (pays only under stated conditions), **[rejected]** (tried and not taken, with the reason), **[proposed]** (not in this tree; future work only). Proposals are never stated as measurements.

1. Question, scope, method

Question. What optimizations make this repository’s elliptic-curve arithmetic fast, at each layer from field operations to fleet clients, and what did each one measure?

Scope. The C core (`src/`, `include/`, `cuda/`: the $p < 2^{64}$ prime-field library and its Pollard-rho-centric consumers), the GPU backend, the FPGA ECC2K-130 core, the ECC2K-130 CUDA client, and the Sage binary-curve batch APIs. The Rust `suite/` arithmetic (complete formulas, Montgomery ladders, GLV, F4 kernels) is mapped only by pointer: it is a separate study surface with its own benches. No IC1 IDs, no curve registry changes, no runs.

Method. Read the implementation files and their measured docs (`docs/ALGORITHMS.md`, `docs/BENCHMARKS.md`, `docs/GPU.md`, `fpga/README.md`, `ecc2k130/README.md`, `experiments/sage-binary-*/README.md`) at the base commit and extracted each optimization with its file evidence and, where one is recorded, its measured speedup. Numbers below are quoted, never recomputed; the bar chart plots them verbatim.

2. How to read the graph

- **Figure 1 (layers)** is the five-layer stack every optimization belongs to: FIELD → POINT → SCALAR → ENGINE → FLEET.
- **Figure 2 (techniques)** puts each layer in one cluster with its techniques (white boxes) and effects: green = measured on this tree, yellow = conditional or off by default, red = rejected or explicitly not-a-measurement. Dashed edges are cross-layer uses (batched stepping, fixed-base starts, the shared distinguished-point protocol).
- **Figure 3 (speedups)** plots the headline measured ratios. All are primary-suite figures from committed docs; Sage ratios are separate per-step comparisons and must not be multiplied into a cumulative claim.

3. Layer by layer

3.1 FIELD: GF(p), $p < 2^{64}$ (`src/modarith.c`)

- **64-bit Montgomery multiplication [structural]**. The field multiply is Montgomery form throughout (`ca_mont_mul`, `ca_mont_sqr`); point code holds residues in Montgomery form so conversions happen once at the boundary (`src/group_ec.c` stores `w[0..1]` in Montgomery form).
 - **M-FIELD1 [measured]**: 5.9 ns per 61-bit multiply, 169.4 Mops/s (`docs/BENCHMARKS.md`, `ca_bench_ops`).
- **Float-seeded integer sqrt [structural]**. `ca_isqrt` starts from `__builtin_sqrtl` and fixes up at most a couple of units; used by BSGS table sizing and kangaroo/grumpy parameter selection.

3.2 POINT: affine Weierstrass (`src/group_ec.c`)

- **Montgomery simultaneous inversion [structural]**. `ec_batch_op` inverts one prefix product for n additions at ~ 3 extra multiplications each, with infinity/doubling/inverse-pair cases inline so the common path stays branch-light. Batch width 256 in the benches.

Elliptic-curve arithmetic optimizations: layer overview (see full technique graph for evidence)

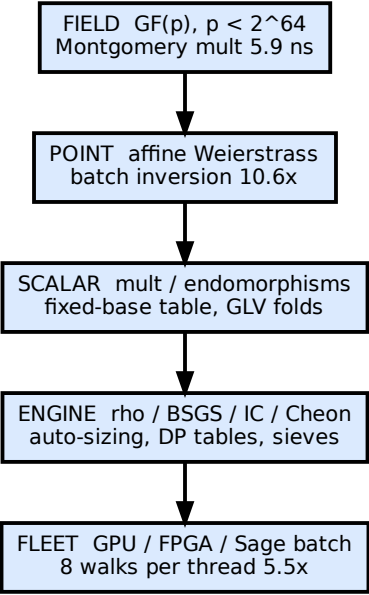


Figure 1: Layer overview. Five arithmetic layers, each with its headline optimization.

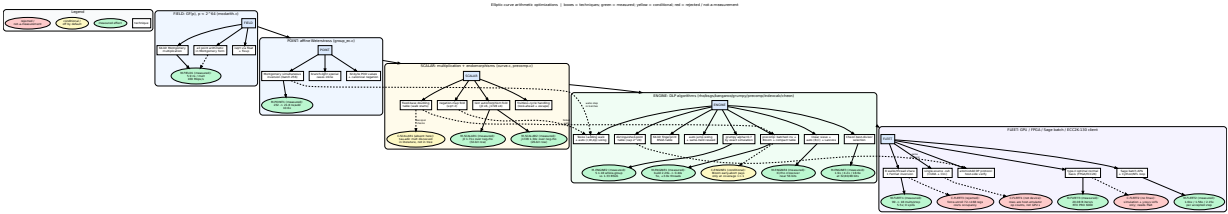


Figure 2: Full technique graph. Green is measured on this tree; yellow is conditional; red is rejected or not-a-measurement. See the SVG for zoomable text.

- **M-POINT1 [measured]**: 232.2 ns per affine add with a private inversion down to 21.8 ns batched – **10.6x** (docs/BENCHMARKS.md). This is what the rho and kangaroo walks step with.
- **32-byte POD values + canonical negation [structural]**. Points are plain values ($w[0]=x$, $w[1]=y$, $w[2]=\text{infinity}$); the rho negation map canonicalizes by smaller Montgomery y , shrinking the search space by 2.

3.3 SCALAR: multiplication + endomorphisms (src/curve.c, src/precomp.c)

- **Fixed-base doubling table [structural]**. Precomputation walk starts from $a \cdot \text{base}$ as the sum over set bits against $G_{\text{pow}}[b] = 2^b \cdot \text{base}$ instead of a generic double-and-add (src/precomp.c).
- **Negation-map fold [structural]**. Default rho already walks $\{Y, -Y\}$ classes (gain $\sqrt{2}$ over plain rho); fruitless 2-cycles handled by the look-ahead rule (rate $1/(2r)$ down to $\sim 1/(2r)^2$) and longer cycles by deterministic doubling escape (Bernstein-Lange-Schwabe).
- **GLV automorphism fold [structural]**. `ca_curve_detect` recognizes the two prime-field CM families and dispatches automatically: $j=0$ ($\psi(x,y)=(\beta x, -y)$, order 6) and $j=1728$ ($\psi(x,y)=(-x, iy)$, order 4); the eigenvalue branch is fixed empirically on a subgroup point so the wrong λ cannot be selected. Expected gain over negation rho is $\sqrt{m/2}$: 1.73 for j_0 , 1.41 for j_{1728} .
 - **M-SCALAR1 [measured]**: j_0 row at 32 bits: 1.71x over negation rho.
 - **M-SCALAR2 [measured]**: j_{1728} row at 26 bits: 1.34x over negation rho (20 instances/row, so $\pm 20\%$ noise; one j_{1728} row lands at 0.94x).
- **Tau-adic multiplication: absent on this line [proposed]**. No `ec_tau.c` or tau experiment exists at the base commit. A width- w tau-NAF in $Z[\omega]$ with Jacobian tau arithmetic is the natural next scalar optimization for j_0 rho setup; any claim for it needs paired online wall times plus an isolated replay per docs/ISOLATED_BENCHMARKS.md.

3.4 ENGINE: DLP algorithms

- **Teske r-adding walk + auto-sizing (src/rho.c) [structural]**. r, W, dp_bits are chosen from n and thread count so multiplier tables, walk start-ups ($\sim 3 \log_2 n$ ops each), and DP counts ($\sim 32/\text{walk}$, table capped at 2^{24} entries) stay balanced; walks step in batches through one `ca_group_batch_op`; walks running $24 \cdot 2^{dp_bits}$ steps without a DP restart out of DP-less cycles.
- **BSGS fingerprint table [structural]**. 64-bit element fingerprints at 24 bytes per baby step; every match verified by scalar multiplication so collisions cost retries, never wrong answers. Default $m = \text{ceil}(\sqrt{w})$.
- **Kangaroo auto jumps + reseed [structural]**. Mean jump $\sim (\text{kangaroos}) \cdot \sqrt{\text{width}}/4$ (van Oorschot-Wiener), powers of two; tame and wild herds share distinguished points; same-herd collisions re-seed. Measured $S = 1.4 \dots 2.3$, fastest per operation because it stores almost nothing.
- **Grumpy alpha = 0.7 [structural + measured]**. The step $m = \alpha \cdot \sqrt{N}$ was tuned by exact exponent-space simulation then confirmed with 400 real instances per setting.
 - **M-ENGINE2 [measured]**: $S = 1.18$ whole-group vs 1.33 for interleaved BSGS (the known $4/3$ figure); intervals 1.78.
- **Precomputation build chain [structural + measured]**. Fixed-base starts, one batched inversion per W curve additions, threads over a lock-free Bloom plus per-thread endpoint buffers merged at the end, early abort, and a compact sorted 16-byte/entry table ($\sim 4x$ smaller than the hash table).
 - **M-ENGINE1 [measured]**: 36-bit curve build 2.26 s (naive) to 0.44 s (optimized, 1 core) – **5x**; 4 threads build it in 0.11 s (**3.9x** more). Fitted exponents confirm the theory: build $n^{0.672}$ ($R^2 0.99$), online $n^{0.198} n^{0.347}$ (noisy: small counts plus constant starts).
 - **C-ENGINE1 [conditional]**: Bloom early-abort pays only at deliberate over-coverage (2^{17} group at coverage 32: 121k to 77k ops, $\sim 2x$ stored endpoints); at default coverage 1 merges are negligible and it is off.
- **Linear sieve + auto (B,C) + Lanczos (src/indexcalc.c) [structural]**. Coppersmith-Odlyzko-Schroeppel ($H+c1$) ($H+c2$) sieve with per- q roots and trial division of survivors; (B,C) defaults from a bit-length table; factor-base logs for $q \leq 2^{24}$ computed directly by Pohlig-Hellman (skipping tiny-field linear algebra); larger q via structured Gauss + Lanczos on $A^T D A$ with 128-bit accumulators (one % per row), dense fallback, Hensel lift, CRT, and per-log verification before use.

- **M-ENGINE4 [measured]**: crossover with rho near 56 bits (1.3 s IC vs ~1.5 s rho there); the sieve is ~free and Lanczos dominates (quadratic in unknowns).
- **Cheon best-divisor selection [structural + measured]**. `ca_cheon_best_divisor` minimizes $2\sqrt{(p-1)/d} + 2\sqrt{d}$ exponentiations.
 - **M-ENGINE3 [measured]**: 1.9x / 4.2x / 18.6x over rho in group ops at 32/40/48 bits (30x/480x in steps at 32/48 bits).
- **Pohlig-Hellman dispatcher [structural]**. Per-prime-order digit DLP goes to brute force (tiny), BSGS ($q \leq 2^{36}$), rho (whole group), or kangaroo (interval).

3.5 FLEET: GPU / FPGA / Sage batch / ECC2K-130 client

- **8 walks per GPU thread sharing 1 Fermat inversion (cuda/ca_device.cuh) [structural]**. Extended Euclid branches on data, so the device inverts by Fermat (~96 mults); simultaneous inversion across `CA_GPU_W=8` walks (compile-time, fixed local allocation) cuts a step from $96+3=99$ to $96/8+3+3=18$ mults.
 - **M-FLEET1 [measured]**: **5.5x** per step; kernel holds 736 B/thread local memory with **0 spills** at `sm_70-sm_90` (4/8/16 walks tried).
 - **C-FLEET3 [rejected]**: forcing full unroll moves nothing into registers and more than doubles register use (72 to 168 on `sm_70`), costing occupancy; loops stay rolled.
- **Single-source kernel + DP protocol [structural]**. The `.cu` is valid CUDA C++ and C11 via one macro plus `__umul64hi/__int128` selection, so the host emulator runs the identical body rather than a reimplementation. Walks keep (Y, a, b) on-device; DPs are published by `atomicAdd` into a bounded buffer (overflow drops points, never wrong answers); the host drains, solves $(b-b')x = a'-a$ (negation-sign and gcd branches), and verifies by scalar multiplication; same- (a, b) rediscovery sets per-walk restart flags. Launches cost one small upload + kernel + DP-proportional download.
 - **C-FLEET1 [not-a-measurement]**: committed GPU rows are host-emulator op counts ($S \sim 1.3$ – 2.6 , matching CPU rho as van Oorschot-Wiener predicts), portable as counts but saying nothing about device throughput.
- **Type-II optimal normal basis, FPGA + ECC2K-130 (fpga/, ecc2k130/) [structural]**. For $F_{2^{131}}$, $2 \cdot 131 + 1 = 263$ prime with $\text{ord}_{263}(2) = 131$ gives basis $b_i = g^i + g^{-i}$: products are cyclic convolutions (digit-serial shift-and-xor), squaring is a wire permutation (Frobenius free), and Hamming weight is basis-invariant (iteration j and DP test read the held vector with no conversion). The C golden model checks basis, product rule, squaring permutation, and curve identification by derivation.
 - **M-FLEET3 [measured]**: table walk at **20.08 B iterations/s** on one RTX PRO 6000 Blackwell (throughput result; different iteration function from the campaign sigma walk, so no shared collisions).
 - **C-FLEET2 [not-a-measurement]**: FPGA has simulation + yosys 4-LUT areas only – no vendor place-and-route, hence no fmax/utilization/points-per-second.
- **Sage binary batch APIs (experiments/sage-binary-*) [structural]**. `add_pairs/add_cartesian` batch independent additions with one inversion of nonzero denominators (bounded 1024-blocks, inputs validated once, ordinary Sage outputs); NTL fields use a compiled Cython loop with reusable native temporaries plus shared point-construction setup; Koblitz `frobenius_points` uses native NTL squaring. Other representations keep the Python fallback; arithmetic is variable-time.
 - **M-FLEET2 [measured, per-step, ARM64 shared host]**: native field arithmetic **1.61x** (1.53x confirmation), shared point construction **1.56x** (1.82x), native Frobenius **2.15x** (2.31x). Separate comparisons; the docs forbid multiplying them into a cumulative claim.
- **Rust suite/ arithmetic (pointer only)**. Complete addition formulas, Montgomery ladders, GLV, F4 kernels, and pair tables live under `suite/` with their own benches; not surveyed here beyond noting the surface.

4. What did not pay (negative findings)

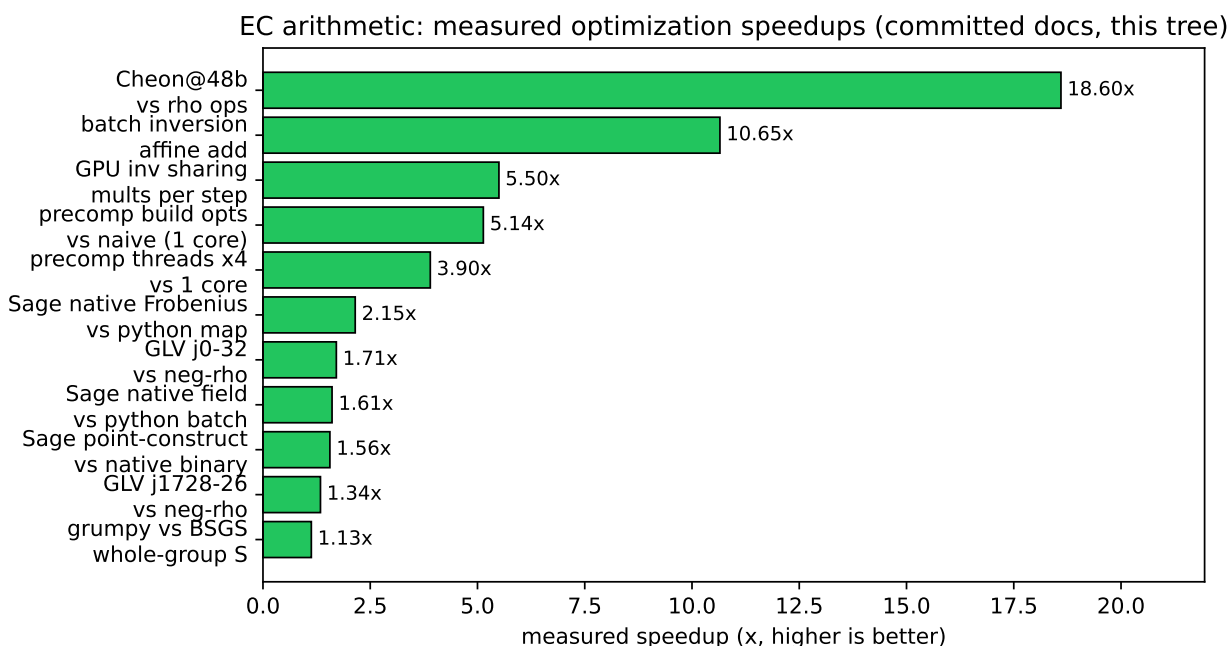
1. **Bloom early-abort at natural coverage [rejected by default]**. Only wins under deliberate over-coverage (§3.4, C-ENGINE1); the default coverage-1 build leaves it off.

2. **GPU loop unrolling [rejected]**. Full unroll more than doubles registers with nothing moved into them (§3.5, C-FLEET3); rolled loops + 0 spills win.
3. **Emulator seconds [not-a-measurement]**. Committed GPU rows carry op counts only; device throughput needs a card (C-FLEET1).
4. **FPGA rates [not-a-measurement]**. No fmax without place-and-route (C-FLEET2).
5. **Tau-adic scalar mult [proposed, absent here]**. The highest-value next scalar optimization for j0 rho setup; needs paired online walls plus an isolated replay before any speedup claim (C-SCALAR1).

5. Quantitative companion (summaries, not new measurements)

Headline measured ratios, plotted verbatim in Figure 3:

optimization	ratio	source
batch inversion, affine add	10.6x	docs/BENCHMARKS.md
GPU inversion sharing, mults/step	5.5x	docs/GPU.md
precomp build opts, 1 core	5x (2.26 s to 0.44 s)	docs/BENCHMARKS.md
precomp threads x4	3.9x	docs/BENCHMARKS.md
GLV j0-32 vs neg-rho	1.71x	docs/BENCHMARKS.md
GLV j1728-26 vs neg-rho	1.34x	docs/BENCHMARKS.md
Cheon@48b vs rho ops	18.6x	docs/BENCHMARKS.md
grumpy vs BSGS whole-group S	1.13x	docs/ALGORITHMS.md
Sage native field / point-construct / Frobenius	1.61x / 1.56x / 2.15x	sage-binary-batch/README.md



Primary-suite figures; Sage ratios must not be multiplied into cumulative-vs-stock claims. Not new measurements.

Figure 3: Measured optimization speedups. Primary-suite figures from committed docs; Sage ratios are per-step and must not be multiplied. Not new measurements.

6. Canonical graphs checked (why they remain current)

This change adds a technique survey; it asserts no new curve, isogeny edge, factor base, run, or speedup, so no canonical measurement graph moves. Checked:

- `experiments/ic-assumption-graph/` (PR #382): different topic (IC assumptions vs arithmetic techniques); unaffected.
- `experiments/volcano-ic/figures/`, `experiments/ic-bench/` `baselines`, `paper/` `figures`: no cited conclusion or figure changes.
- `experiments/ic-candidate-catalog/curves.yaml`, `isogeny_routes.json`: no new nodes or edges.
- `experiments/fb-archive/index.csv`: no new rows; the `ic-bench` archive gate is unaffected (this directory matches no workflow path filter).

7. Uncertainty and open checks

- All numbers are quoted from committed docs measured on other hosts (4-core x86-64 container for the C core, ARM64 shared host for Sage, RTX PRO 6000 for the ECC2K-130 table walk). Operation counts and ratios are the portable part; seconds are not. Statistical noise is +/-20-30% on small rho rows (noted per table).
- Open, highest value first: (1) tau-adic scalar path for j0 with paired online walls + isolated replay; (2) portable SIMD dispatch for the Montgomery multiply with per-class reporting per repo performance rules;
(3) closing the FPGA loop with vendor place-and-route for fmax; (4) a `suite/-arithmetic` companion survey with its bench evidence.
- No independent red-team review was run for this survey. A claim-changing follow-up (new optimization, new speedup) needs the repo's baseline, isolation, and review gates, not just an edit to this graph.

8. Reproduce

```
cd experiments/ec-arithmetic-optimizations
dot -Tsvg opt_layers.dot -o figures/opt_layers.svg
dot -Tpdf opt_layers.dot -o figures/opt_layers.pdf
dot -Tsvg ec_optimizations.dot -o figures/ec_optimizations.svg
dot -Tpdf ec_optimizations.dot -o figures/ec_optimizations.pdf
python3 figures.py # speedup chart -> figures/speedups.pdf/.png
pandoc REPORT-2026-10-06.md -o REPORT-2026-10-06.pdf \
  --pdf-engine=pdflatex -V geometry:margin=1in -V colorlinks:true
```

Graphviz 2.43.0, pandoc 3.1.3, pdfTeX 3.141592653-2.6.1.40.25 (TeX Live 2023/Debian) on this VM. Inspect the SVG in a browser (zoomable) and the PDF for embedded-vector readability before citing.

9. References

Repo sources read in full at the base commit:

- `docs/ALGORITHMS.md`, `docs/BENCHMARKS.md`, `docs/GPU.md`.
- `src/group_ec.c`, `src/modarith.c`, `src/curve.c`, `src/rho.c`, `src/bsgs.c`, `src/kangaroo.c`, `src/grumpy.c`, `src/precomp.c`, `src/indexcalc.c`, `src/cheon.c`, `src/pohlig.c`; `include/cryptanalysis/ca_group.h`, `ca_modarith.h`, `ca_curve.h`.
- `cuda/ca_device.cuh`, `cuda/rho_kernel.cu`, `src/gpu_rho.c`, `src/gpu_emulate.c`.
- `fpga/README.md`, `ecc2k130/README.md`.
- `experiments/sage-binary-batch/README.md`, `experiments/sage-binary-native-addition/README.md`, `experiments/sage-binary-point-construction/README.md`, `experiments/sage-binary-native-frobenius/README.md`.

Literature (recalled pointers, not re-read here): Montgomery 1985 (simultaneous inversion / multiplication); Teske 2001 (r-adding walks); van Oorschot-Wiener 1999 (parallel collision search); Bernstein-Lange-Schwabe

2011 (negation map); Gallant-Lambert-Vanstone 2001 (GLV); Bernstein-Lange 2012/2013 (precomputation); Coppersmith-Odlyzko-Schroeppel 1986 (linear sieve); LaMacchia-Odlyzko 1990 (Lanczos); Cheon 2006/2010.